

Module 12: Polymorphism & Virtual Functions

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

This module resolves the cliffhanger from Module 11 and delivers the actual payoff of inheritance: a single loop that correctly handles a mixed collection of different derived types. The object-slicing demonstration is the emotional and conceptual center of the lesson - build the whole session around making that failure visible before showing the one-keyword fix.

Suggested Duration: 75–90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain object slicing and reproduce it deliberately.
- Use virtual to enable dynamic dispatch through a base pointer/reference.
- Use override to catch mismatched override signatures at compile time.
- Build and correctly iterate over a vector of base-class pointers holding mixed derived types.
- Explain why a polymorphic base class needs a virtual destructor.

Prerequisites

- Module 11 (Inheritance) - this module directly continues that lesson's running example.

Materials & Instructor Prep

- 12-polymorphism/examples/01_slicing_problem.cpp and 02_virtual_fixes_it.cpp preloaded, ready to run back-to-back for direct comparison.

Key Vocabulary

Term	Definition
Object slicing	Copying a derived object into a base-typed variable/parameter, losing everything the derived class added.

Term	Definition
virtual	A qualifier on a base class method telling the compiler to dispatch calls based on the object's actual type at runtime, not the pointer/reference's declared type.
Dynamic dispatch	Deciding which override of a method to run at runtime, based on the real object type.
override	A keyword confirming a method is intended to override a virtual base method - a compile-time safety check, not what enables polymorphism itself.
Virtual destructor	A destructor marked virtual so deleting a derived object through a base pointer cleans up the whole object correctly.
Abstract class / pure virtual function	A class with a <code>= 0</code> method, making it impossible to instantiate directly - only its derived classes can be created.

Lesson Outline & Instructor Talking Points

1. Resolve the cliffhanger: demonstrate slicing live (20 min)

- Run examples/01_slicing_problem.cpp. Before revealing the output, ask the class to predict what `printIt(r)` will print - most will confidently say the Resistor version.
- Reveal the wrong unit in the output. Let the surprise land fully before explaining WHY: `printIt(Component c)` takes its parameter BY VALUE, copying only the Component part of `r` - the Resistor-ness is gone.

Instructor note: *This demonstration is the single most important moment in the module. Do not rush to the fix - let the class sit with 'why did this happen' for a minute before moving on.*

2. The fix: virtual (20 min)

- Add `virtual` to Component's `describe()` and change `printIt` to take a reference instead of a value. Run examples/02_virtual_fixes_it.cpp and show the correct output now.
- Name BOTH changes explicitly as necessary: `virtual` alone does not fix slicing if you still pass by value - the object must already be a reference/pointer for there to be anything left to dispatch on.

Instructor note: *This is a common half-understanding to watch for: some students will think adding `virtual` alone is sufficient. Explicitly quiz this misconception in the room before moving on.*

3. override as a safety net (10 min)

- Introduce override and explain it's a compile-time check, not what makes polymorphism work. Demonstrate: deliberately misspell an override's name or parameter type and show the compiler catches it when override is used, vs. silently creating an unrelated method when it isn't.

4. The payoff: a polymorphic collection (25 min)

- Run examples/03_polymorphic_collection.cpp. Build the `vector<Component*>` live, pushing back a Resistor, Capacitor, and Inductor with new.
- Loop over it once with `c->describe()` and let the class watch each object correctly print its OWN version - explicitly connect back: 'this loop never checks what kind of Component each one is. It just works, for every current and future derived type.'

Instructor note: *This is the payoff moment for the whole Module 11-12 arc. Give it real time and let it feel like a genuine 'that's powerful' moment, not just another code example.*

5. Virtual destructors (10 min)

- State the rule plainly: any class meant to be used as a polymorphic base needs a virtual destructor, even an empty one. Show what silently goes wrong (a resource leak) without it, at a conceptual level appropriate for this stage.

Common Student Misconceptions

Misconception	How to Address It
"Adding virtual to a method is enough, regardless of how the object is passed around."	virtual only helps once you're working through a pointer or reference - passing by value still slices, virtual or not, because the copy has already lost the derived part.
"override makes a method virtual."	override only checks that a method correctly matches an EXISTING virtual method in the base class - virtual itself must still be declared on the base.
"A <code>vector<Component*></code> holding different derived types is somehow unusual or advanced."	Reframe: this is the single most common reason polymorphism exists at all - handling a mixed collection uniformly is the everyday use case, not an edge case.

Exercises & Module Project

Exercise: 12-polymorphism/exercises/exercise1.cpp - an Animal/Dog/Cat polymorphic vector.

Project (three tracks): Generic - Shape collection with total area; EE - Circuit Impedance Calculator (a `vector<Component*>` summing impedance at a frequency); Biomedical - Unified Patient Monitor (a `vector<MedicalDevice*>` printing one dashboard).

Facilitation guidance: In every track, check specifically that the base class's shared method and destructor are BOTH marked virtual, that override appears on every derived override, and that every new'd pointer is deleted before the program ends - this project is exactly where a missed virtual destructor first has real (if invisible at this scale) consequences.

Discussion & Comprehension Check Questions

1. Why does `printIt(Component c)` slice the object, but `printIt(Component& c)` does not?

Guidance: Pass-by-value copies only the Component part of the argument; pass-by-reference refers to the actual, complete original object, so there's a real derived object left for virtual dispatch to work on.

2. What's the practical reason to build a `vector<Base*>` instead of separate `vector<Derived1>`, `vector<Derived2>`, ... collections?

Guidance: A single mixed collection lets you write ONE loop/algorithm that correctly handles every current and future derived type, instead of duplicating logic per type.

Assessment Ideas

- Predict-then-run: give a short snippet mixing pass-by-value and pass-by-reference calls to a virtual method, and have students predict which calls slice before running it.

Answer Key Notes

See 12-polymorphism/exercises/solutions/exercise1_solution.cpp.

Wrap-Up & Bridge to Next Module

Bridge forward: "Your components/devices/shapes can now be handled uniformly - but they still can't be added together or compared with natural syntax like numbers can. Next module fixes that with operator overloading."

Instructor Tips

- If time is short, protect the slicing demonstration (step 1) and the polymorphic-collection payoff (step 4) over the abstract-class preview in the README - those two are the load-bearing moments of this module.